Administrivia

HW4 due tonight

HW5 section tomorrow (RBTree)

OH moved to 2pm!

A quick aside: Transferring ownership

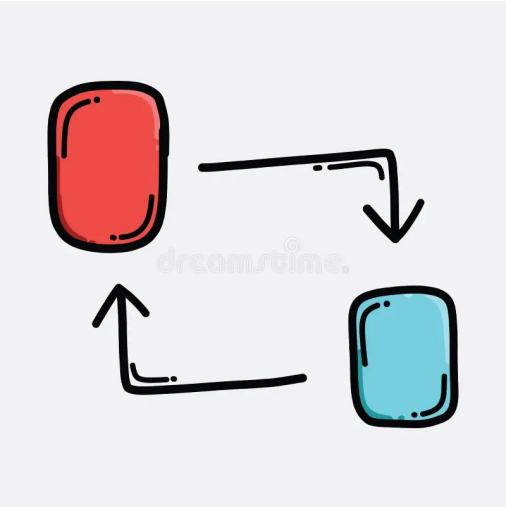
What if we want x and y to swap values?

There is a std::mem function for this! std::mem::swap

```
let mut x = 5;
let mut y = 42;

std::mem::swap(&mut x, &mut y);

assert_eq!(42, x);
assert_eq!(5, y);
```



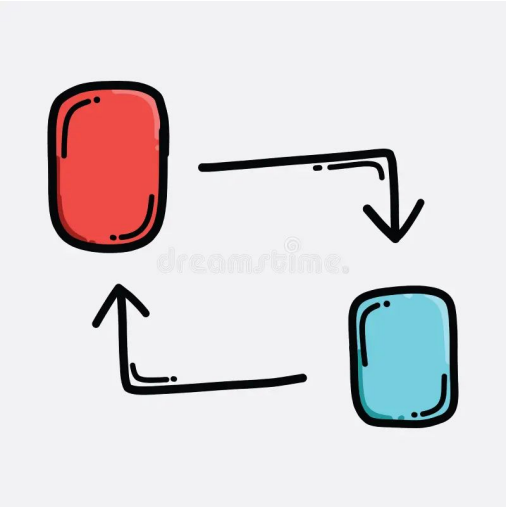
A quick aside: Transferring ownership

Why is this useful?

You can never have an undefined struct element in Rust!

```
let x = Rectangle {contents: vec![1,2]}
let y = Rectangle {contents: vec![3,4,5]}

// swap x and y's contents
let tmp = y.contents; // moved all of y!
// now y.contents is undefined
y.contents = x.contents; // cannot use y!
x.contents = tmp;
```



A quick aside: Transferring ownership

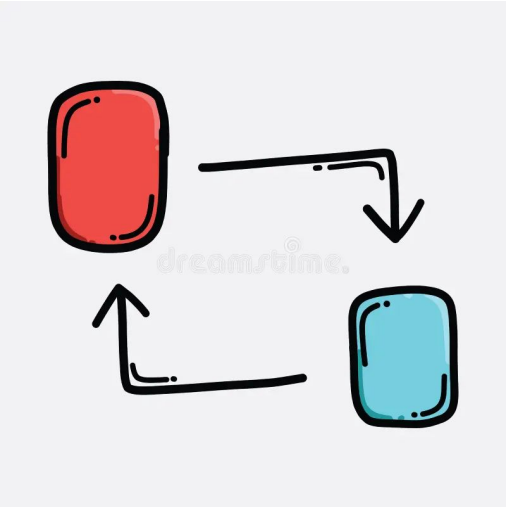
Why is this useful?

You can never have an undefined struct element in Rust!

```
let x = Rectangle {contents: vec![1,2]}
let y = Rectangle {contents: vec![3,4,5]}

// swap x and y's contents

std::mem::swap(&mut x.contents, &mut y.contents);
```



Moving on: Pointer Types

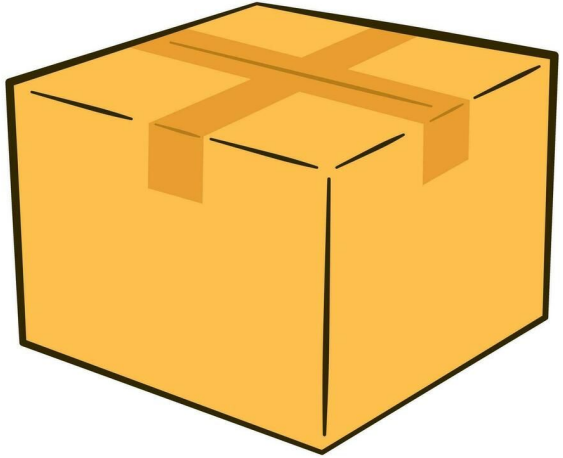
	Owned	Borrowed
Value	T	
Pointer		&T

Moving on: Pointer Types

	Owned	Borrowed
Value	T	
Pointer	Box<T>	&T

Box<T>

- T on the Heap
- Can obtain a mutable reference to inner value
- Still owned by a stack variable
  - So that the compiler knows when to free!



If a slice is a &[T]...

What is a Box<[T]>?

- Also a slice! But one that you own!

Demo

- Linked List (Box<T>)

Quiz (11:14-11:20)

Select all that apply.

```rust
struct Vec2 {
    x: f64,
    y: f64,
}
impl Vec2 {
    fn scale(self: Vec2, factor: f64) -> Vec2 {
        Vec2 {
            x: self.x * factor,
            y: self.y * factor,
        }
    }
}
fn main() {
    let v = Vec2 { x: -1.0, y: 5.0 };
    v.scale(2.0);
    println!("The upscaled Vec2 is: {} {}", v.x, v.y);
}
```

Smart pointers, shared ownership

- Rc<T> defines a reference-counted heap-allocated T

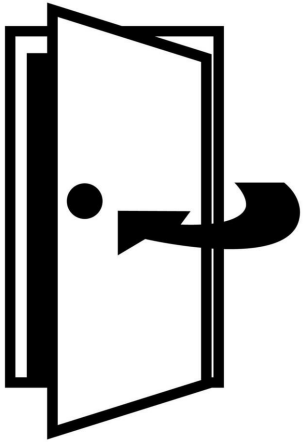
```
let x = Rc::new(334);

let y = x.clone();

// both x and y can access the number 334

// 334 will only get deallocated when both x and y are dropped
```

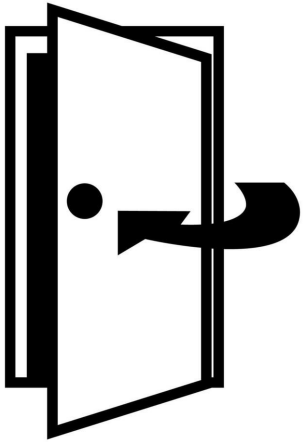
Closures vs Functions



A closure is a function that can also capture local variables

Closures are bound to variables!

Closures vs Functions



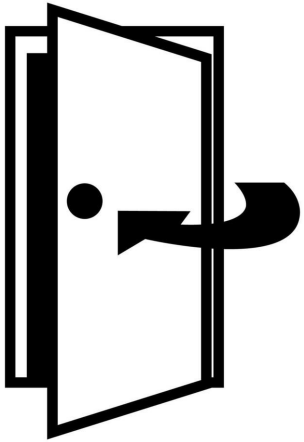
```
fn  add_one_v1   (x: u32) -> u32 { x + 1 }

let add_one_v2 = |x: u32| -> u32 { x + 1 };

let add_one_v3 = |x|             { x + 1 };

let add_one_v4 = |x|               x + 1  ;
```

Closures can capture local variables!

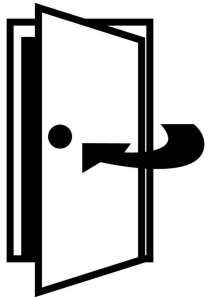


```
let mut list = [
    Rectangle { width: 10, height: 1 },
    Rectangle { width: 3, height: 5 },
    Rectangle { width: 7, height: 12 },
];

let mut num_sort_operations = 0;
list.sort_by_key(|r| {
    num_sort_operations += 1;
    r.width
});
```

Rust figures out to mutably borrow this!

Closures can take ownership of local variables!



```
let data = vec![1, 2, 3];

std::thread::spawn(move || {
    println!("captured {data:?} by value")
}).join().unwrap();

// data was moved to the spawned thread, so we cannot use it
here
```

Move keyword tells Rust to move ownership of local variables into this closure!

Dynamic Dispatch

So far, we've only learned about static dispatch, e.g.

```
fn introduce<T: Animal>(animal: T) {
    animal.say_hi();
}
```

If you pass in a T that implements Animal, the compiler will dispatch which "copy" of introduce to call at compile time.

Dynamic Dispatch

Dynamic dispatch refers to deciding which to run at runtime

Small overhead: each struct must maintain a table of how to dispatch!

- An Animal has to remember that it is a Cow to know which say_hi to use

Why might we need dynamic dispatch?

Dynamic Dispatch Example

```
let v: Vec<Animal>;
```

Animal is a trait, not a type!

Dynamic Dispatch Example

```
let v: Vec<dyn Animal>;
```

Elements in v dynamically dispatch to the correct impl Animal for T block

Dynamic Dispatch Example

```
let v: Vec<dyn Animal>;
```

different Animal impls can be different sizes!

Dynamic Dispatch Example

```
let v: Vec<&dyn Animal>;
```

We need to put dyn Animal behind a pointer to ensure it is Sized

Dynamic Dispatch Example

```
let v: Vec<Box<dyn Animal>>;
```

Dynamic Dispatch Example

```
let v: Vec<Rc<dyn Animal>>;
```

Dynamic Dispatch: C++/Java vs Rust

```
Foo* (Pointer to derived object)
    |
    v
+--------+---------+---------+--------+
| Vtable*| field_1 | field_2 | ...    |
+--------+---------+---------+--------+
    |
    v
+----------+-----+
| method_1 | ... |
+----------+-----+
```

```
&dyn Foo (Trait Object)
+--------+----------+
| Data * | Vtable * |
+--------+----------+
    |        |
    |        v
    |     +---------------+--------+----------+-------+
    |     | drop_in_place | size   | method_1 | ...   |
    |     +---------------+--------+----------+-------+
    v
+---------+---------+--------+
| field_1 | field_2 | ...    |
+---------+---------+--------+
```

Visibility

Are there private/public variables and methods and functions? Yes!

Everything private by default

`pub` keyword to make something accessible outside of the current module

Module

A logical grouping of functions, structs, traits, impl blocks, and even other modules

Purpose is to define a visibility scope

Typically, a file implements one module

Each folder is a module that contains many modules

- mod.rs “re-exports” items from within that folder

Crates, packages libs, and main

A package is one “project”, and consists of one *binary crate* and some *library crates*

- Binary crate: a set of modules that includes a main function
  - root module: main.rs, or tests/my_test.rs
- Library crate: a set of modules that does not include a main function
  - root module: lib.rs

Most functionality should be placed in library modules!

Safe Rust

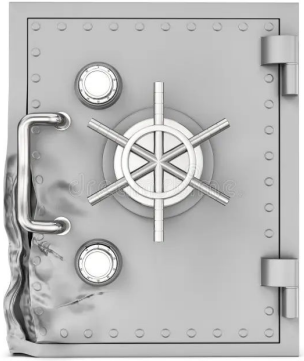
What does safe Rust give us?

- No dangling / null references
- No segfaults
- No use-after-free
- No data races
- Type safety

Unsafe Rust

Allows you to use “raw pointers”, references that are not guaranteed to be valid

You must ensure that these raw pointers point to valid data, or U.B.!



You should never need to use unsafe Rust in this course.

Generally, you’ll only need it to interoperate with C libraries.